In [ ]:
```python
import os
import numpy as np

# Torch Import
import torch
import torch.nn as nn
import torch.nn.functional as F


import torchvision
import torchvision.transforms as transforms

from torch.utils.data import DataLoader, Dataset

from sklearn.neighbors import NearestNeighbors

from typing import List, Tuple

from sklearn.decomposition import PCA

import hdbscan
```

In [ ]:
```python
# Reproducibility
torch.manual_seed(0)
np.random.seed(0)

## MODIFIED
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False
## END OF MODIFICATION


SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
os.chdir(SCRIPT_DIR)


class ResNetEncd(nn.Module):
    def __init__(self, base="resnet18", proj_dim=128):
        super().__init__()

        backbone = getattr(torchvision.models, base)(pretrained=False)
        self.backbone = nn.Sequential(*list(backbone.children())[:-1])
        feat_dim = backbone.fc.in_features

        self.projection = nn.Sequential(
            nn.Linear(feat_dim, feat_dim),
            nn.ReLU(inplace=True),
            nn.Linear(feat_dim, proj_dim)
        )

    def forward(self, x, return_projection: bool = True):
        feat = self.backbone(x)
        feat = torch.flatten(feat, 1)
        if return_projection:
            proj = self.projection(feat)
            return feat, proj
        return feat
```

```
    def represent(self, x):
        return self.forward(x, return_projection=False)
```

In [ ]:

```
# Depreciated Use - kept for logging reasons
class ConstructiveNTXent(nn.Module):
    def __init__(self, temperature: float = 0.2):
        super().__init__()
        self.temperature = temperature

    def forward_pass(self, z_a: torch.Tensor, z_b : torch.Tensor) -> torch.Tensc

        batch = z_a.size(0)
        z = torch.cat([z_a, z_b], dim=0)

        sim_matrix = torch.matmul(z, z.T) / self.temperature

        diag_mask = torch.eye(2 * batch ,  device=z.device).bool()

        sim_matrix.masked_fill_(diag_mask, -9e15)

        positives = torch.arange(batch, 2 * batch, device = z.device)
        labels = positives.repeat(2) # Cross-entropy target to be satisfied

        loss = F.cross_entropy(sim_matrix, labels) # Cross-Entropy loss funct.
        return loss


# NT-Xent Loss  - CONSTRASTIVE NTXent
class ContrastiveNTXent(nn.Module):
    def __init__(self, temperature : float = 0.2):
        super().__init__()
        self.temperature = temperature


    def forward(self, z_a: torch.Tensor, z_b : torch.Tensor) -> torch.Tensor:

        batch = z_a.size(0)
        z = torch.cat([z_a, z_b], dim =0 )
        z = F.normalize(z, dim=1)


        sim_matrix = torch.matmul(z, z.T) / self.temperature

        diag_mask = torch.eye(2 * batch, device= z.device).bool()
        sim_matrix.masked_fill_(diag_mask, -9e15)

        labels = torch.arange(batch, device=z.device) # 1D-tensor
        labels = torch.cat([labels + batch, labels], dim=0)

        loss = F.cross_entropy(sim_matrix, labels)
        return loss



# VVVVV Augmentation of data Below VVVVV

# 2-Crop Transformation
class TC_Transform:
    def __init__(self):
        self.aug = transforms.Compose([
```

```
            transforms.RandomResizedCrop(32, scale=(0.2,1.0)),
            transforms.RandomHorizontalFlip(),
            transforms.ColorJitter(0.4, 0.4, 0.4, 0.1),
            transforms.RandomGrayscale(p=0.2),
            transforms.ToTensor(),
            transforms.Normalize((0.4914, 0.4822, 0.4465),
                                 (0.2470, 0.2435, 0.2616)),
        ])


    def __call__(self, img):
        return self.aug(img), self.aug(img)
```

In [ ]:

```
def compute_typicality(features: np.ndarray, k: int = 20) -> np.ndarray:

    nbrs = NearestNeighbors(n_neighbors=min(k + 1, len(features)), algorithm='au
    distances, _ = nbrs.kneighbors(features)  # distances: [N, k+1]
    distances = distances[:, 1:]
    avg_sq_dist = (distances ** 2).mean(axis=1)
    typicality = 1.0 / (avg_sq_dist + 1e-12)
    return typicality


# Multi-Round Typiclust
def typiclust_selection(
    features: np.ndarray,
    initial_labeled: List[int],
    budget_total: int,
    batch_size_per_round: int,
    k_typicality: int = 20,
    random_state: int = 0,
) -> List[int]:


    ###  MODIFIED FROM ORIGINAL ###

    # Modified Distance Parameters
    lambda_ = 0.01

    # DBSCAN Parameters
    #db = DBSCAN(eps=1.2, min_samples=10).fit(features)
    #cluster_labels = db.labels_

    # HDBSCAN
    clusterer = hdbscan.HDBSCAN(
        min_cluster_size = 30,
        min_samples =5,
        metric="euclidean"
    ).fit(features)

    cluster_labels = clusterer.labels_

    unique, counts = np.unique(cluster_labels, return_counts=True)
    n_noise = int(np.sum(cluster_labels == -1))
    n_clusters = int((unique != -1).sum())
    print(f"[HDBSCAN] clusters={n_clusters}, noise={n_noise}, total={len(cluster
    print("Top cluster sizes:", sorted([(c, s) for c, s in zip(unique, counts) i

    # Save metadata for documentation ( HDBSCAN )
    meta = {
```

```
        "min_cluster_size": 30,
        "min_samples": 5,
        "lambda": lambda_,
        "n_clusters": n_clusters,
        "n_noise": n_noise
    }
    os.makedirs("budget_results/meta", exist_ok=True)
    np.save("budget_results/meta/hdbscan_meta.npy", meta)



    ###  END OF MODIFICATION FROM ORIGINAL ###

    N = features.shape[0]
    rng = np.random.RandomState(random_state)

    # Reduction of noise points ( -1 labels)
    valid_mask = cluster_labels != -1
    valid_indices = np.where(valid_mask)[0]

    labeled = set(initial_labeled)
    unlabeled = set(range(N)) - labeled

    typicality = compute_typicality(features, k=k_typicality)

    while len(labeled) < budget_total:


        # DEBUG PRINT
        print(f"[LOOP] labeled = {len(labeled)}, unlabled = {len(unlabeled)}")

        # clusters that DO  [ NOT ]  contain a labeled point
        unique_clusters = np.unique(cluster_labels[valid_mask])

        # DEBUG PRINT
        print(f"[Clusters] unique_clust = {len(unique_clusters)}")

        if unique_clusters.size == 0:
            print("Warning: HDBSCAN found no clusters (all points noise). Increa
            break

        cluster_has_label = {c: False for c in unique_clusters}

        for idx in labeled:
            c = cluster_labels[idx]
            if c != -1:
                cluster_has_label[c] = True

        # DEBUG PRINT
        uncovered_clusters = [c for c in unique_clusters if not cluster_has_labe
        print(f"[Uncovrd] uncovered_clusters = {len(uncovered_clusters)}")

        if len(uncovered_clusters) == 0:
            break

        new_indices = []

        for c in uncovered_clusters:
            if len(labeled) + len(new_indices) >= budget_total:
                break
```

```
            cluster_idx = np.where(cluster_labels == c)[0]
            cluster_idx = [i for i in cluster_idx if i in unlabeled]

            if not cluster_idx:
                continue

            # Diversity penalised scoring
            scores = typicality[cluster_idx] - lambda_ * distance_to_selected(cl
            best_local = cluster_idx[np.argmax(scores)]
            new_indices.append(best_local)

        for idx in new_indices:
            labeled.add(idx)
            unlabeled.remove(idx)

        if not new_indices:
            break



    return sorted(labeled)
    ###  END OF MODIFICATIONS FROM ORIGINAL ###
```

In [ ]:

```
    ###  MODIFIED FROM ORIGINAL ###
def distance_to_selected(indices, features, labeled):
    if len(labeled) == 0:
        return np.zeros(len(indices))

    selected_feats = features[list(labeled)]
    dists = np.linalg.norm(features[indices][:, None] - selected_feats[None, :],
    return dists.min(axis=1)

    ###  END OF MODIFICATIONS FROM ORIGINAL ###



class TwoCropCIFAR_10(Dataset):
    def __init__(self, root: str, train:bool, transform):
        self.dataset = torchvision.datasets.CIFAR10(
            root = root,
            train= train,
            download = True,
            transform = None
        )
        self.transform = transform

    def __len__(self):
        return len(self.dataset)

    def __getitem__(self, idx):
        img, _ = self.dataset[idx]
        x1, x2 = self.transform(img)
        return x1, x2, idx



def train_self_supervised(
    encoder : ResNetEncd,
    dataset: Dataset,
```

```
    device: torch.device,
    batch_size: int = 256,
    epochs: int = 200,
    lr: float = 1e-3,
    resume_path : str = None
) -> ResNetEncd:
    loader = DataLoader(
        dataset,   # TO DO - ADD DATASET & BATCH_SIZE integration
        batch_size=batch_size,
        shuffle=True,
        num_workers=4,
        drop_last=True
    )

    ssl_losses = []


    encoder = encoder.to(device)
    loss_fn = ContrastiveNTXent(temperature=0.2).to(device)
    optimizer = torch.optim.Adam(encoder.parameters(), lr=lr)


    start_epoch = 0
    if resume_path is not None and os.path.exists(resume_path):
        print(f"Resuming SSL training from checkpoint: {resume_path}")
        checkpoint = torch.load(resume_path, map_location=device)
        encoder.load_state_dict(checkpoint["encoder"])
        optimizer.load_state_dict(checkpoint["optimizer"])
        start_epoch = checkpoint["epoch"] + 1
        print(f"Resumed from epoch {start_epoch}")


    encoder.train()
    for epoch in range(epochs):
        running_loss = 0.0
        for x1, x2, _ in loader:
            x1= x1.to(device)
            x2 = x2.to(device)

            _, z1 = encoder(x1, return_projection=True)

            _, z2 = encoder(x2, return_projection=True)

            loss = loss_fn(z1,z2)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            running_loss += loss.item() * x1.size(0)

        epoch_loss = running_loss / len(loader.dataset)
        print(f"Epoch {epoch+1}/{epochs} - SSL LOSS : {epoch_loss:.4f}")

        ssl_losses.append(epoch_loss)

        # Checkpoint every 40 epochs
        if (epoch+1)%40 == 0:
            os.makedirs("budget_results/ssl_checkpoints", exist_ok=True)
            ckpt_path = f"budget_results/ssl_checkpoints/ssl_epoch_{epoch+1}.pth
            torch.save(encoder.state_dict(), ckpt_path)
```

```
            print(f"Checkpoint Saved [ CKLP ] at {epoch+1} -> {ckpt_path}")

    np.save("budget_results/ssl_loss.npy", np.array(ssl_losses))
    return encoder
```

In [ ]:

```
# Extraction of Features
def extract_features(
        encoder : ResNetEncd,
        dataset : torchvision.datasets.CIFAR10,
        device : torch.device,
        batch_size : int = 256
) -> np.ndarray:
    loader = DataLoader(
        dataset,
        batch_size=batch_size,
        shuffle=False,
        num_workers =4
    )

    encoder.eval()
    all_features = []

    with torch.no_grad():
        for x, _ in loader:
            x = x.to(device)
            features = encoder.represent(x)
            all_features.append(features.cpu().numpy())

    features = np.concatenate(all_features, axis=0)
    return features



def run_pipeline_selection(
    data_root: str = "./data",
    budget_total: int = 100,
    batch_size_per_round: int = 10,
    ssl_epochs: int = 200,
):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    ssl_transform = TC_Transform()
    ssl_dataset = TwoCropCIFAR_10(root=data_root, train=True, transform=ssl_tran

    encoder = ResNetEncd(base="resnet18", proj_dim=128)
    encoder = train_self_supervised(
        encoder=encoder,
        dataset=ssl_dataset,
        device=device,
        batch_size=256,
        epochs=ssl_epochs,
        lr=1e-3,
    )

    base_transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize(
            (0.4914, 0.4822, 0.4465),
            (0.2470, 0.2435, 0.2616)
        ),
```

```
    ])
    base_dataset = torchvision.datasets.CIFAR10(
        root=data_root,
        train=True,
        download=True,
        transform=base_transform
    )

    features = extract_features(
        encoder=encoder,
        dataset=base_dataset,
        device=device,
        batch_size=256
    )

    # Reduce dimensionality for easier computation
    pca = PCA(n_components=50)
    features = pca.fit_transform(features)

    #Multi-round TypiClust
    labeled_indices = typiclust_selection(
        features=features,
        initial_labeled=[],
        budget_total=budget_total,
        batch_size_per_round=batch_size_per_round,
        k_typicality=20,
        random_state=0,
    )

    os.makedirs("budget_results", exist_ok=True)
    np.save("budget_results/typiclust_selection.npy", np.array(labeled_indices))
    print(f"Selected {len(labeled_indices)} labeled points (single-round)")

    return labeled_indices
```

In [ ]:

```
def generate_and_save_typiclust_selections(
        budgets=[10,20,40,80],
        batch_size_per_round=10,
        ssl_epochs=500,
        data_root="./data"

):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    ssl_transform = TC_Transform()
    ssl_dataset = TwoCropCIFAR_10(root=data_root, train=True, transform=ssl_tran

    encoder = ResNetEncd(base="resnet18", proj_dim=128)
    encoder = train_self_supervised(
        encoder=encoder,
        dataset=ssl_dataset,
        device=device,
        batch_size=256,
        epochs=ssl_epochs,
        lr=1e-3,
    )

    base_transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize(
```

```
            (0.4914, 0.4822, 0.4465),
            (0.2470, 0.2435, 0.2616)
        ),
    ])
    base_dataset = torchvision.datasets.CIFAR10(
        root=data_root,
        train=True,
        download=True,
        transform=base_transform
    )

    features = extract_features(
        encoder=encoder,
        dataset=base_dataset,
        device=device,
        batch_size=256
    )

    os.makedirs("budget_results", exist_ok=True)

    for B in budgets:
        labeled_indices = typiclust_selection(
            features=features,
            initial_labeled=[],
            budget_total=B,
            batch_size_per_round=batch_size_per_round,
            k_typicality=20,
            random_state=0,
        )

        np.save(f"budget_results/typiclust_B{B}.npy", np.array(labeled_indices))
        print(f"Saved TypiClust selection for B={B} → budget_results/typiclust_B


def execute_HDBSCAN_PROCESS(
        features_path="modified_budget_results/features.npy",
        checkpoint_path="modified_budget_results/ssl_checkpoints/ssl_epoch_480.p
        budgets=[10,20,40,80],
        lambda_=0.01
):




    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    # Load your trained encoder
    encoder = ResNetEncd(base="resnet18", proj_dim=128)
    encoder.load_state_dict(torch.load(checkpoint_path, map_location=device))
    encoder = encoder.to(device)
    encoder.eval()

    # Load extracted features
    features = np.load(features_path)

    pca = PCA(n_components=50)
    features = pca.fit_transform(features)

    results_dir = "modified_budget_results/hdbscan_results"
    os.makedirs(results_dir, exist_ok=True)
```

```
    for B in budgets:
        labeled_indices = typiclust_selection(
            features=features,
            initial_labeled=[],
            budget_total=B,
            batch_size_per_round=10,
            k_typicality=20,
            random_state=0,
        )

        out_path = f"{results_dir}/typiclust_HDBSCAN_B{B}.npy"
        np.save(out_path, np.array(labeled_indices))

        print(f"HDBSCAN selection saved for B={B} → {out_path}")
```

In [ ]:

```
if __name__ == '__main__':
    #generate_and_save_typiclust_selections(budgets=[10], ssl_epochs=5) # For te
    run_pipeline_selection(ssl_epochs=500) # Run pipeline()


    # To generate 500 epochs & perform DBSCAN
    #generate_and_save_typiclust_selections(ssl_epochs=500) # reduce due to time

    # To perform strictly only DBSCAN
    execute_HDBSCAN_PROCESS(
        features_path="modified_budget_results/features.npy",
        checkpoint_path="modified_budget_results/ssl_checkpoints/ssl_epoch_480.p
        budgets=[10,20,40,80],
        #eps=200.0, # No longer needed for HDBSCAN
        lambda_=0.01
    )
```